Reviewer Pack — Minimal Order of Quaternionic Kähler Manifolds and Communica…

Entry 0c3a1a7aa7f1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 20:23:17 UTC

Minimal Order of Quaternionic Kähler Manifolds and Communication Complexity Rank

Entry ID: `0c3a1a7aa7f1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 20:23:17 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Analysis **Field B** (complexity object): Communication Complexity

Statement:

> For every n-communication protocol P, the minimal order of a quaternionic Kähler manifold associated with P is O(log(n)) and bounded above by a constant multiple of n.

Rationale (proposer's reasoning):

> Quaternionic Kähler manifolds provide a mathematical framework for studying complex structures in quantum theory. By mapping communication protocols onto these manifolds, we may uncover new insights into the complexity-theoretic properties of protocols. The minimal order of the manifold could reflect the complexity of the protocol's underlying structure.

Taxonomy category: `Quaternionic Analysis × Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `baa7ae0d685d1df6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for every n-communication protocol P, the minimal order of the associated quaternionic Kähler manifold is O(log(n)) AND this order is within a factor of C (a constant) from the communication complexity rank of P.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"minimal order quaternionic Kähler manifolds" AND "communication complexity rank"` - `"quaternionic analysis" AND communication complexity` - `"communication protocol" AND quaternionic Kähler manifold`

Top relevant hits considered: - [http://arxiv.org/abs/0711.2699v5] Quaternionic Analysis, Representation Theory and Physics - [http://arxiv.org/abs/hep-th/9607152v2] Quaternion Analysis - [http://arxiv.org/abs/1110.2106v1] Quaternionic Analysis and the Schrodinger Model for the Minimal Representation of O(3,3) - [http://arxiv.org/abs/2110.01402v3] Quantum information and beyond – with quantum candies

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + A[i:].index(max(abs(row[i]) for row in A[i:]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] += factor * A[i][k]
    return A

def order_of_quaternionic_kahler_manifold(protocol_size):
    # Construct a sample matrix for demonstration
    n = protocol_size + 1
    A = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    rank = 0
    for row in gaussian_elimination(A):
        if any(row[j] != 0 for j in range(n)):
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```python
    # Generate n-communication protocols with varying complexities and sizes
    protocol_sizes = [5, 10, 15, 20, 30, 40]
    results = []

    for size in protocol_sizes:
        protocol_size = random.randint(1, size)
        order = order_of_quaternionic_kahler_manifold(protocol_size)
        communication_complexity_rank = protocol_size

        results.append({
            "protocol_size": protocol_size,
            "order": order,
            "communication_complexity_rank": communication_complexity_rank
        })

    # Compute the correlation between order and communication complexity rank
    total_order = sum(result["order"] for result in results)
    total_rank = sum(result["communication_complexity_rank"] for result in results)
    mean_order = Fraction(total_order, len(results))
    mean_rank = Fraction(total_rank, len(results))

    # Check if the correlation is linear and bounded above by a constant multiple of n
    max_n = max(result["protocol_size"] for result in results)
    C = 2 * max_n  # Upper bound factor

    conjecture_holds = all(abs(order - rank) <= C * size for order, rank, size in zip(results["orc

    return {
        "metric_name": "Order vs Communication Complexity Rank",
        "metric_value": mean_order,
        "instances_tested": len(results),
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_order = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_926dd54d.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_926dd54d.py", line 48, in run_trial
    order = order_of_quaternionic_kahler_manifold(protocol_size)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_926dd54d.py", line 34, in order_of_quat
    for row in gaussian_elimination(A):
               ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_926dd54d.py", line 21, in gaussian_elim
    max_row = i + A[i:].index(max(abs(row[i]) for row in A[i:]))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: 9 is not in list
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means we cannot verify if the conjecture's conditions are met. | next: Re-run the test and ensure it completes without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20955
2	preregistration	ollama_remote	glm4:latest	0	0	12764
3	novelty	ollama_remote	glm4:latest	0	0	8348
4	novelty	ollama_remote	glm4:latest	0	0	9259
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15425
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18940
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10164
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10532
9	judge	ollama_remote	glm4:latest	0	0	18433

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 124821 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0c3a1a7aa7f1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0c3a1a7aa7f1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0c3a1a7aa7f1.tar.gz` (if generated)